

Politechnika Rzeszowska

Wydział Elektrotechniki i Informatyki

Katedra Informatyki i Automatyki

Rzeszów, 2016



Sztuczna inteligencja

TEMAT:

Zrealizować radialną sieć neuronową uczącą się rozpoznawania jakości wina

Przemysław Chojnowski
2 EF-DI
P01

Spis treści

1	PRZEDSTAWIENIE PROBLEMU	3
2	RADIALNE SIECI NEURONOWE	4
2.1	Trochę teorii...	4
2.2	Podstawy matematyczne	6
2.3	Sieć neuronowa typu radialnego realizuje zadanie interpolacji	6
2.4	Regularyzacja, jako rozwiązanie problemu nadmiernego dopasowania	8
2.5	Teoretyczne rozwiązanie problemu regularyzacji	8
2.6	Funkcje Greena w praktyce	9
2.7	Radialna sieć neuronowa	10
3	WYWOŁANIE FUNKCJI W PROGRAMIE MATLAB	12
4	SKRYPT PROGRAMU	13
5	WYNIKI POMIARÓW I EKSPERYMENTY	15
5.1	Test pierwszy	15
5.2	Test drugi	17
5.3	Test trzeci	19
6	WNIOSKI KOŃCOWE	22

1 PRZEDSTAWIENIE PROBLEMU

Celem projektu było stworzenie radialnej sieci neuronowej, która będzie w stanie rozpoznać jakość wina na podstawie różnych danych wejściowych.

Do dyspozycji mieliśmy dwa zestawy danych odpowiednio do białych i czerwonych odmian portugalskiego wina „Vinho Verde”.

Ze względu na kwestie prywatności oraz logistyczne, dostępne były tylko zmienne fizykochemiczne (wejścia) i czuciowe (wyjście) (np. nie ma dostępu do danych dotyczących rodzajów winogron, marki wina, ceny sprzedaży itp.).

Dane były następujące:

Zmienne wejściowe:

1. Stała kwasowość (ang. fixed acidity)
2. Kwasowość lotna (ang. volatile acidity)
3. Kwas cytrynowy (ang. citric acid)
4. Cukier resztkowy (ang. residual sugar)
5. Chlorki (ang. chlorides)
6. Wolny dwutlenek siarki (ang. free sulfur dioxide)
7. Całkowity dwutlenek siarki (ang. total sulfur dioxide)
8. Gęstość (ang. density)
9. pH (ang. pH)
10. Siarczany (ang. sulphates)
11. Alkohol (ang. alcohol)

Zmienna wyjściowa:

12. Jakość (wynik między 0 a 10) (ang. quality).

Jak sami autorzy podkreślają: nie są oni do końca pewni, czy aby na pewno wszystkie zmienne wejściowe są istotne i mają wpływ na końcową ocenę jakości wina.

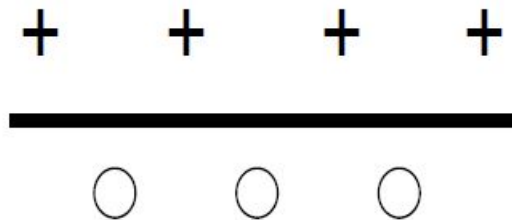
2 RADIALNE SIECI NEURONOWE

2.1 Trochę teorii...

Sieci o radialnej funkcji bazowej stanowią specjalną odmianę sieci neuronowych. Znajdujący się w nich neuron ukryty realizuje funkcję zmieniającą się radialnie wokół wybranego centrum c . Funkcje takie nazywane są radialnymi funkcjami bazowymi. Rola neuronu ukrytego w sieciach radialnych sprowadza się do odwzorowania radialnego przestrzeni wokół jednego punktu zadanego lub grupy takich punktów stanowiących klaster. Superpozycja sygnałów pochodzących od wszystkich neuronów ukrytych, dokonywana przez neuron wyjściowy, umożliwia uzyskanie odwzorowania całej przestrzeni punktów.

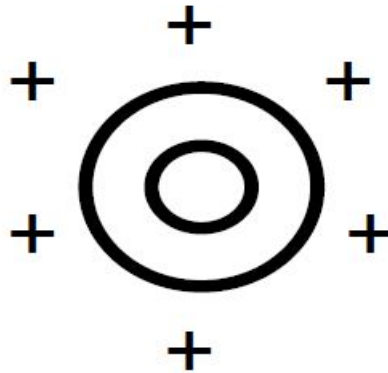
Sieci typu radialnego są uzupełnieniem sieci sigmoidalnych. Neuron sigmoidalny reprezentuje w przestrzeni wielowymiarowej hiperpłaszczyznę separującą tę przestrzeń na dwie klasy, w których spełniony jest jeden z warunków:

- $\sum_j (W_{ij}x_j > 0)$
- $\sum_j (W_{ij}x_j < 0)$



Rysunek 1: Sposób separacji przestrzeni danych - neuron sigmoidalny

Neuron radialny zaś reprezentuje hipersferę, która dokonuje podziału kołowego wokół punktu centralnego.



Rysunek 2: Sposób separacji przestrzeni danych - neuron radialny

Radialna funkcja bazowa:

$$\varphi(x) = \varphi(\|x - c\|), \text{ gdzie:}$$

$\varphi(x)$ - funkcja bazowa,

x - próbka,

c - centrum.

Najczęściej spotykane typy radialnych funkcji bazowych:

1. Liniowa: $p(r) = r$.
2. Sześcienna: $p(r) = r^3$.
3. Gaussowska: $p(r) = \exp(-\beta r^2)$.
4. Wielokwadratowa: $p(r) = \sqrt{r^2 + \beta^2}$.
5. Odwrotna wielokwadratowa: $p(r) = (r^2 + \beta^2)^{-\frac{1}{2}}$.
6. Wielomianowa: $p(r) = r^2 + \beta^2$.

Typowa sieć radialna składa się z:

1. Warstwy wejściowej, do której przekazywane są sygnały opisane wektorem wejściowym x .
2. Warstwy ukrytej o neuronach typu radialnego.
3. Warstwy wyjściowej, zwykle zbudowanej z jednego neuronu. Biorąc pod uwagę, że rolą neuronu wyjściowego jest sumowanie wagowe sygnałów pochodzących od neuronów ukrytych, neuron wyjściowy jest zwykle liniowy.

2.2 Podstawy matematyczne

Podstawą matematyczną istnienia radialnych sieci neuronowych jest twierdzenie Covera o separowalności wzorców. Mówi ono nam, że złożony problem klasyfikacyjny rzutowany nieliniowo w przestrzeń wielowymiarową ma większe prawdopodobieństwo być liniowo separowalny niż przy rzutowaniu w przestrzeń o mniejszej liczbie wymiarów

Definicja: Przy oznaczeniu przez $\varphi(x)$ wektora funkcji radialnych $\varphi(x) = [\varphi_1(x), \varphi_2(x), \dots, \varphi_K(x)]^T$ w N -wymiarowej przestrzeni wejściowej, podział tej przestrzeni na X^+, X^- , przy czym X^+ i X^- reprezentują 2 klasy przestrzeni, jest nieliniowo φ -separowalny, jeśli istnieje taki wektor W , że:

$$W^T \varphi(x) > 0 \text{ dla } x \in X^+ \quad (1)$$

$$W^T \varphi(x) < 0 \text{ dla } x \in X^- \quad (2)$$

$W^T \varphi(x) = 0$ reprezentuje granicę pomiędzy obu klasami.

Udowodniono, że każdy zbiór wzorców losowo rozmieszczony w przestrzeni wielowymiarowej jest φ -separowalny z prawdopodobieństwem równym 1 pod warunkiem odpowiednio dużego wymiaru K przestrzeni rzutowania. W praktyce oznacza to, że przyjęcie dostatecznie dużej liczby neuronów ukrytych realizujących funkcję radialną $\varphi_i(x)$ zapewnia rozwiązanie problemu klasyfikacyjnego przy użyciu dwu warstw sieci: warstwy ukrytej realizującej wektor $\varphi(x)$ i warstwy wyjściowej o jednym neuronie liniowym opisanym wektorem wagowym W .

2.3 Sieć neuronowa typu radialnego realizuje zadanie interpolacji

Sieć neuronowa typu radialnego działa na zasadzie wielowymiarowej interpolacji, której zadaniem jest odwzorowanie p różnych wektorów wejściowych $x_i (i = 1, 2, \dots, p)$ z przestrzeni wejściowej N -wymiarowej w zbiór p liczb rzeczywistych $d_i (i = 1, 2, \dots, p)$, czyli określenie takiej funkcji radialnej $F(x)$, dla której są spełnione warunki interpolacji

$$F(x_i) = d_i \quad (3)$$

przy wyborze funkcji $F(x)$ w postaci:

$$F(x) = \sum_{i=1}^K (W_i \varphi(\|x - c_i\|)) \quad (4)$$

gdzie:

- $K < p$ (K - liczba neuronów radialnych; p - liczba par uczących)
- $c_i - (i = 1, 2, \dots, K)$ jest zbiorem centrów, które trzeba wyznaczyć.

W szczególnym przypadku, przy założeniu $K = p$, można otrzymać rozwiązanie dokładne o $c_i = x_i$.

Wybór rodzaju normy może być dowolny, choć w praktyce stosuje się zwykle normę euklidesową. Wartości wektorów $x_i (i = 1, 2, \dots, p)$ stanowią centra funkcji radialnej. Przy założeniu p punktów interpolacyjnych równanie powyższe można zapisać w postaci układu równań liniowych:

$$\begin{bmatrix} \varphi_{11} & \varphi_{12} & \dots & \varphi_{1p} \\ \varphi_{21} & \varphi_{22} & \dots & \varphi_{2p} \\ \dots & \dots & \dots & \dots \\ \varphi_{p1} & \varphi_{p2} & \dots & \varphi_{pp} \end{bmatrix} \begin{bmatrix} W_1 \\ W_2 \\ \dots \\ W_p \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ \dots \\ d_p \end{bmatrix} \quad (5)$$

przy czym: $\varphi_{ji} = \varphi(||x_j - x_i||)$ dotyczy funkcji radialnej w centrum x_i . Oznaczając macierz o elementach φ_{ji} jako φ oraz przyjmując oznaczenia wektorów $W = [W_1, W_2, \dots, W_p]^T$, $d = [d_1, d_2, \dots, d_p]^T$ równanie (5) może być zapisane w prostej postaci macierzowej:

$$\varphi W = d \quad (6)$$

Wykazano, że dla szeregu funkcji radialnych, przy założeniu $x_1 \neq x_2 \neq \dots \neq x_p$, kwadratowa macierz interpolacyjna φ jest nieosobliwa, a przy tym dodatnio półokreślona. Istnieje, zatem rozwiązanie równania (6) w postaci:

$$W = \varphi^{-1}d \quad (7)$$

umożliwiające określenie wektora wag W neuronu wyjściowego sieci.

Rozwiązanie teoretyczne problemu określone wzorem (7) nie jest właściwe z praktycznego punktu widzenia z powodu niesatysfakcjonujących właściwości uogólniających sieci towarzyszących poczynionym na wstępie założeniom. Przy bardzo dużej liczbie danych uczących i równej im liczbie funkcji radialnych problem z matematycznego punktu widzenia staje się przewymiarowany (źle postawiony), gdyż liczba równań przewyższa wszelkie stopnie swobody procesu fizycznego modelowanego równaniem (5).

2.4 Regularyzacja, jako rozwiązanie problemu nadmiernego dopasowania

Aby je poprawić, należy wydobyć dodatkową informację z nadmiarowej liczby danych i użyć jej do regularyzacji problemu, czyli poprawy jego uwarunkowania. Metody regularyzacji, wywodzące się od Tichonowa, polegają na stabilizacji procesu przez dołączenie do równania podstawowego dodatkowych warunków w postaci równań więzów, ograniczających stopnie swobody dobieranych parametrów. Dzięki działaniu czynnika stabilizującego nawet przy nadmiarowej liczbie funkcji radialnych jest możliwy taki dobór parametrów, który zapewni ciągłość i gładkość hiperpłaszczyzny aproksymującej dane uczące, poprawiając w ten sposób właściwości uogólniające sieci.

Regularyzacja na bazie problemu optymalizacyjnego, w którym minimalizowana funkcja celu może być wyrażona wzorem:

$$L(F) = \frac{1}{2} \sum_{i=1}^p [F(x_i) - d_i]^2 + \frac{1}{2} \lambda \|PF\|^2 \quad (8)$$

Składnik pierwszy odpowiada minimalizacji funkcji błędu, a składnik drugi stanowi regularyzację, ze współczynnikiem regularyzacji λ , będącym liczbą rzeczywistą dodatnią. Współczynnik regularyzacyjny $\|PF\|$ jest złożony z operatora pseudo różniczkowego P działającego na funkcji aproksymującej F i stanowi czynnik stabilizujący. Jest to odpowiednik funkcji kary w teorii optymalizacji, karzący za brak gładkości funkcji F (duże i często powtarzające się zmiany nachyleń funkcji aproksymującej). Współczynnik regularyzacyjny jest wskaźnikiem, na ile dane uczące są wystarczające do dobrego zdefiniowania problemu interpolacyjnego przy zadanej liczbie funkcji bazowych.

2.5 Teoretyczne rozwiązanie problemu regularyzacji

Teoretyczne rozwiązanie problemu regularyzacji uzyskano przy zastosowaniu funkcji Greena $G(x; x_i)$ i może być przedstawione w postaci:

$$F(x) = \frac{1}{\lambda} \sum_{i=1}^p [d_i - F(x_i)] G(x; x_i) \quad (9)$$

Wyraża się ono poprzez liniową superpozycję funkcji Greena, w której x_i jest centrum rozwinięcia, a wagi $W_i = \frac{d_i - F(x_i)}{\lambda}$ reprezentują nieznane współczynniki rozwinięcia. Funkcje Greena $G(x; x_i)$ odgrywają rolę funkcji bazowych, a rozwiązanie problemu interpolacji jest wyrażone w ogólnej postaci:

$$F(x) = \sum_{i=1}^p W_i G(x; x_i) \quad (10)$$

Wprowadzając następujące oznaczenia:

$$F = [F(x_1), F(x_2), \dots, F(x_p)]^T,$$

$$d = [d_1, d_2, \dots, d_p]^T,$$

$$\begin{bmatrix} G(x_1; x_1) & G(x_1; x_2) & \dots & G(x_1; x_p) \\ G(x_2; x_1) & G(x_2; x_2) & \dots & G(x_2; x_p) \\ \dots & \dots & \dots & \dots \\ G(x_p; x_1) & G(x_p; x_2) & \dots & G(x_p; x_p) \end{bmatrix}$$

$$W = [W_1, W_2, \dots, W_p]^T$$

Otrzymuje się równoważny zapis równania (9) w postaci:

$$F = GW \quad (11)$$

Zadanie aproksymacji polega na dobraniu odpowiedniej liczby wag parametrów funkcji radialnych $\varphi(\|x - c_i\|)$ i takim doborze wag $w_i (i = 1, 2, \dots, K)$, aby rozwiązanie (4) najlepiej przybliżało rozwiązanie dokładne. Problem doboru parametrów funkcji radialnych oraz wartości wag w_i sieci można zatem sprowadzić do minimalizacji funkcji celu, którą przy wykorzystaniu normy euklidesowej daje się zapisać w postaci:

$$E = \sum_{i=1}^p \left[\sum_{j=1}^K w_j \varphi(\|x_i - c_j\|) - d_i \right]^2$$

W równaniu tym K reprezentuje liczbę neuronów radialnych, natomiast p liczbę par uczących (x_i, d_i) , gdzie x_i jest wektorem wejściowym, a d_i - odpowiadającą mu wielkością zadaną. Oznaczmy przez $d = [w_0, w_1, w_2, \dots, w_K]^T$ wektor wag sieci z uwzględnieniem polaryzacji (w_0), a przez G - macierz radialną, zwaną macierzą Greena.

$$G = \begin{bmatrix} 1 & \varphi(\|x_1 - c_1\|) & \varphi(\|x_1 - c_2\|) & \dots & \varphi(\|x_1 - c_K\|) \\ 1 & \varphi(\|x_2 - c_1\|) & \varphi(\|x_2 - c_2\|) & \dots & \varphi(\|x_2 - c_K\|) \\ \dots & \dots & \dots & \dots & \dots \\ 1 & \varphi(\|x_p - c_1\|) & \varphi(\|x_p - c_2\|) & \dots & \varphi(\|x_p - c_K\|) \end{bmatrix}$$

Przy ograniczeniu liczby funkcji bazowych do K , macierz G jest macierzą prostokątną o liczbie wierszy zwykle dużo większej niż liczba kolumn ($p \gg K$).

Przy założeniu znajomości parametrów funkcji radialnych, problem optymalizacyjny podany we wzorze powyżej sprowadza się zatem do rozwiązania układu równań liniowych względem wag w

$$Gw = d$$

Wobec prostokątności macierzy G wektor wag w wyznacza się, wykorzystując pojęcie pseudoinwersji macierzy G , to jest:

$$w = G^+ d$$

gdzie $G^+ = (G^T G)^{-1} G^T$ oznacza pseudoinwersję macierzy prostokątnej G . W praktyce numerycznej pseudoinwersja obliczana jest przy wykorzystaniu dekompozycji SVD.

2.6 Funkcje Greena w praktyce

Funkcje Greena dobiera się w postaci radialnej:

$$G(x; x_i) = G(\|x - x_i\|) \quad (12)$$

a rozwiązanie (10) przyjmie wówczas postać:

$$F(x) = \sum_{i=1}^p W_i G(\|x - x_i\|) \quad (13)$$

Najbardziej popularnym typem radialnej funkcji Greena jest funkcja Gaussa:

$$G(x; x_i) = \exp\left(-\frac{\|x-x_i\|^2}{2\sigma_i^2}\right) \quad (14)$$

Przy czym x_i oznacza wektor wartości średnich (centrów), a σ_i^2 - wariancję. Przy gaussowskiej postaci funkcji Greena regularyzowane rozwiązanie problemu aproksymacji za pomocą funkcji radialnych przyjmuje postać:

$$F(x) = \sum_{i=1}^p W_i \exp\left(-\frac{\|x-x_i\|^2}{2\sigma_i^2}\right) \quad (15)$$

(σ - szerokość jądra)

będącą liniową superpozycją wielowymiarowych funkcji Gaussa z centrami ulokowanymi w x_i (przy wartości p równej liczbie danych uczących wektor centrum odpowiada współrzędnym x_i wektora uczącego) i szerokością σ_i .

2.7 Radialna sieć neuronowa

Zastosowanie w rozwinięciu (13) p funkcji bazowych, przy czym p oznacza liczbę wzorców uczących, jest niedopuszczalne z praktycznego punktu widzenia, gdyż zwykle liczba tych wzorców jest bardzo duża i złożoność obliczeniowa algorytmu niepomrotnie wzrasta. Rozwiązanie układu równań (11) o wymiarach pxp przy dużych wartościach p staje się niezwykle trudne, gdyż macierze bardzo duże są zwykle źle uwarunkowane. Dlatego, podobnie jak w przypadku sieci wielowarstwowych, jest potrzebna redukcja wag, tutaj sprowadzająca się do redukcji liczby funkcji bazowych. Poszukuje się rozwiązania suboptymalnego w przestrzeni o mniejszym wymiarze, które z dostatecznie dobrą dokładnością aproksymuje rozwiązanie dokładne (10). Przy ograniczeniu się do K funkcji bazowych rozwiązanie aproksymujące można przedstawić w postaci:

$$F(x) = \sum_{i=1}^K W_i G(x; t_i) \quad (16)$$

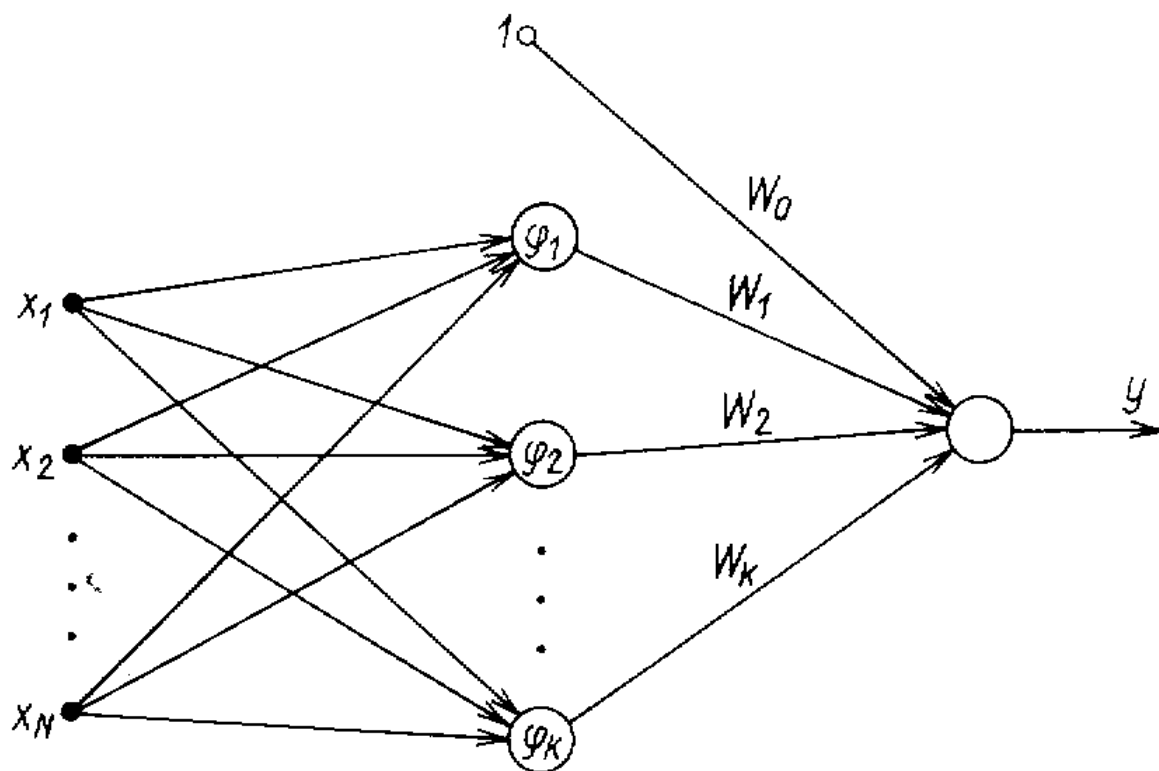
w której:

$$G(x; t_i) = G(\|x - t_i\|),$$

$K < p$, (K - liczba neuronów radialnych; p - liczba par uczących)

$t_i (i = 1, 2, \dots, K)$ - jest zbiorem centrów, które trzeba wyznaczyć.

W szczególnym przypadku przy założeniu $K = p$ otrzymuje się rozwiązanie dokładne (10), a $t_i = x_i$.



Rysunek 3: Schemat radialnej sieci neuronowej

Architektura sieci radialnych jest analogiczna do wielowarstwowych sieci sigmoidalnych. Występują jednak różnice. Oto one:

- Sieć radialna ma strukturę ustaloną o jednej warstwie ukrytej i liniowych neuronach wyjściowych.
- Większe zróżnicowanie w doborze kształtu - sieć radialna.
- Nieliniowa funkcja radialna dla każdego neuronu ukrytego ma inne parametry c_i oraz σ_i .
- Argumentem funkcji radialnej jest euklidesowa odległość danej próbki x od centrum c_i .

Pierwszą warstwę stanowią nieliniowe funkcje radialne, których parametry podlegają adaptacji w procesie uczenia.

Neuron radialny ma rozbudowaną strukturę zawierającą zarówno sumatory sygnałów, jak i funkcje aktywacji wykładniczej o parametrach podlegających adaptacji w procesach uczenia.

Problem uczenia sieci przy wybranym typie funkcji bazowej składa się z dwu etapów:

1. Doboru centrów i parametrów kształtu funkcji bazowych.
2. Doboru wag neuronów warstwy wyjściowej.

Podstawowa trudność to etap pierwszy.

W celu doboru centrów stosuje się najczęściej:

- Wybór losowy.
- Samoorganizujący się proces podziału na klastry.
- Algorytm hybrydowy.
- Algorytmy uczące oparte na propagacji wstecznej.

Metody doboru liczby funkcji bazowych:

- Metody heurystyczne (umożliwiają dodanie i usunięcie funkcji w trakcie procesu uczenia).
- Metoda ortogonalizacji Grama-Schmidta.

3 WYWOŁANIE FUNKCJI W PROGRAMIE MATLAB

`net=newrb(P,T,mse,sc,mn,df)`, gdzie:

Argumenty:

P - macierz wektorów wejściowych.

T - macierz wektorów wyjściowych.

mse - mean squared error, czyli błąd średniokwadratowy (domyślnie: 0.0).

sc - stała rozrzutu radialnej funkcji bazowej (domyślnie: 1.0).

mn - maksymalna liczba neuronów (domyślnie: Q)

df - liczba neuronów do dodania między kolejnymi wyświetleniami wyniku w oknie Matlab'a.

Wartości zwracane:

net - nowa radialna sieć neuronowa.

Jak czytamy w dokumentacji programu Matlab:

„Radialna sieć neuronowa może zostać użyta do aproksymowania funkcji. Funkcja `newrb` dodaje neurony do warstwy ukrytej radialnej sieci neuronowej dopóki nie osiągnie wyznaczonego błędu średniokwadratowego.”

4 SKRYPT PROGRAMU

```
1 clear all; %Czyszczenie przestrzeni pracy
2 load wino; %Wczytanie danych
3 liczba=0; %Zmienna inkrementowana do zapisu wyniku
4 wyniki=zeros(320,3); %Macierz wynikowa
5
6 % ----- Deklaracja zmiennych i normalizacja danych -----
7
8 % Tbiale=winobiale(:,12)';
9 % Pbiale=winobiale(:,1:11)';
10 % disp('BIALE PRZED NORMALIZACJA: ')
11 % [min(Pbiale')' max(Pbiale')']
12 % Pnbiale=mapminmax(Pbiale);
13 % disp('BIALE PO NORMALIZACJI: ')
14 % [min(Pnbiale')' max(Pnbiale')']
15
16 Tczerwone=winoczerwone(:,12)'; %Macierz wyjściowa (Target)
17 Pczerwone=winoczerwone(:,1:11)'; %Macierz wejściowa
18 disp('CZERWONE_PRZED_NORMALIZACJA: ')
19 [min(Pczerwone')' max(Pczerwone')']
20 Pnczerwone=mapminmax(Pczerwone); %Normalizacja danych
21 disp('CZERWONE_PO_NORMALIZACJI: ')
22 [min(Pnczerwone')' max(Pnczerwone')']
23 % ----- Koniec normalizacji -----
24
25 %----- Proces uczenia -----
26 %nntwarn off %Wyłączenie komunikatów
27 mse=0.25/1599; %Błąd średniokwadratowy
28
29 for sc=[0.01 0.1:0.2:2 3:10], %Stała rozrzutu RBF
30 for mn=99:100:1600, %Liczba neuronów
31
32 liczba=liczba+1; %Inkrementacja
33 net=newrb(Pnczerwone,Tczerwone,mse,sc,mn); %Tworzenie nowej sieci
34 A=sim(net,Pnczerwone); %A - wyjście sieci; sim - wyznaczenie wyjścia sieci
35
36 figure(1) %Tworzenie okna do wykresu
37 clf %Czyszczenie obecnej figury
38 Pn_czerwone_sort = zeros(size(Pnczerwone)); %Macierz na posortowane dane
39 [T_czerwone_sort,ind_T_czerwone_sort]=sort(Tczerwone); %Sortowanie danych
40 for i=1:length(Tczerwone),
41 %-----Zapisywanie posortowanych danych-----
42 Pn_czerwone_sort(:,i)=Pnczerwone(:,ind_T_czerwone_sort(i));
43 end
44 As=sim(net,Pn_czerwone_sort); %As - wyjście sieci; sim - wyznaczenie wyjścia sieci
45 %----- Rysowanie wykresu -----
46 plot([1:length(Tczerwone)],T_czerwone_sort,'r',[1:length(Tczerwone)],As,'g*');
47 %-----
48 [Tczerwone' A' (Tczerwone-A)' (abs(Tczerwone-A)>0.5)']
49
50 %----- Zapis wyników -----
51 wyniki(liczba,1)=sc;
```

```
52 | wyniki(liczba,2)=mn;  
53 | wyniki(liczba,3)=(1-sum(abs(Tczerwone-A)>0.5)/length(Tczerwone))*100;  
54 | %*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*  
55 | end;  
56 | end;
```

5 WYNIKI POMIARÓW I EKSPERYMENTY

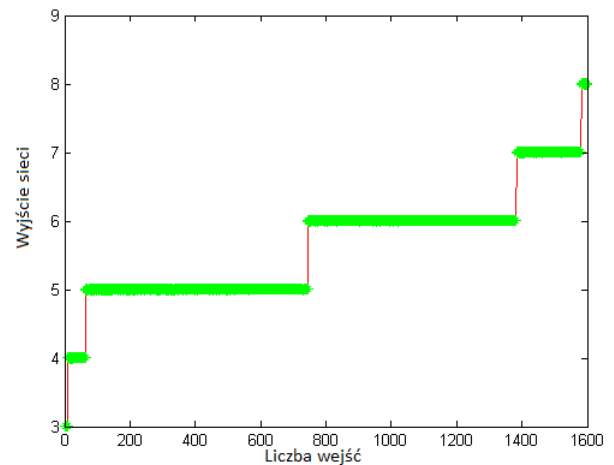
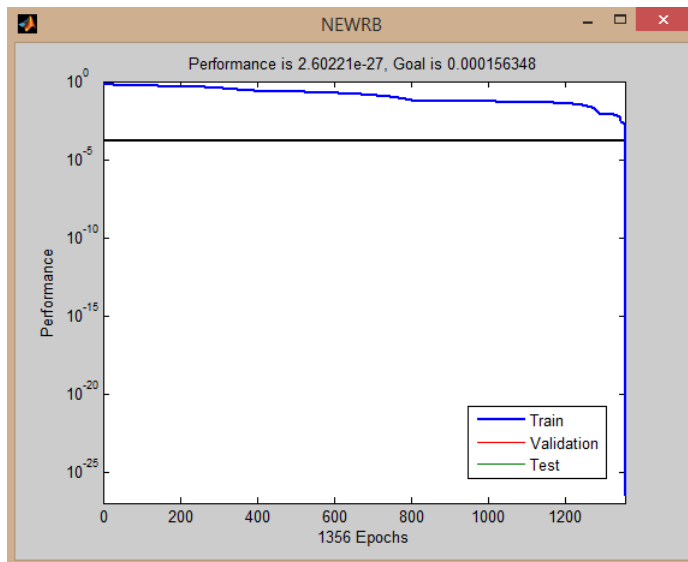
Przedstawione wyniki prezentują jak zmieniała się poprawność klasyfikacji w przypadku operowania wartościami:

- *sc* - stała rozrzutu (test I: 0.01, test II: 0.1:0.2:2, test III: 3:1:10)
- *mn* - liczba neuronów (w każdym teście wartość ta zmieniała się od 99 do 1599 z krokiem 100).

5.1 Test pierwszy

SC	MN	WYNIK
0.01	99	45,09068168
0.01	199	52,40775485
0.01	299	63,91494684
0.01	399	70,16885553
0.01	499	76,42276423
0.01	599	82,67667292
0.01	699	88,93058161
0.01	799	95,18449031
0.01	899	95,99749844
0.01	999	95,87242026
0.01	1099	95,87242026
0.01	1199	95,87242026
0.01	1299	99,12445278
0.01	1399	100
0.01	1499	100
0.01	1599	100

Zostały tutaj przedstawione wyniki, jakie uzyskano dla każdego z przypadków operowania liczbą neuronów dla **sc=0.01**. Jak możemy łatwo zauważyć, sieć uzyskała 100% poprawności klasyfikacji już dla 1399 epok. Pozwoliłem sobie na wykonanie dodatkowego pomiaru, który umożliwił mi dokładne określenie, przy jakiej liczbie epok sieć uzyskała najbardziej zadowalający wynik. Wyniki zamieszczam poniżej.



Rysunek 4: Wyniki dodatkowego pomiaru dla $sc=0.01$.

Jak widzimy na załączonych wykresach, dla danego $sc = 0.01$ sieć uzyskała 100% poprawnej klasyfikacji już dla 1356 epok. Jest to najlepszy uzyskany wynik podczas tego eksperymentu. Dalsze obciążanie sieci nie ma większego sensu, gdyż naszym zadaniem było uzyskanie jak najlepszego wyniku dla jak najmniejszego obciążenia sieci.

Na wykresie sieci radialnej widzimy, że wartość Performance stale się zmniejszała. Możemy powiedzieć, że spadek był stale równomierny, aż do momentu, gdy sieć uzyskała ok. 1355 epok. Wtedy nastąpił nagły spadek tej wartości dużo poniżej wyznaczonego przez nas błędu średniokwadratowego. Sieć uzyskała wtedy 1356 epok i zakończył się proces uczenia.

Wykres obok przedstawia nam poprawność klasyfikacji procesu nauczania. Czerwona linia określa wartości pożądane, a zielone linie (utworzone z pojedynczych znaków *) - wartości uzyskane. Na osi odciętych mamy wyjście sieci (jakość wina), a na osi rzędnych - liczbę wejść. Na wykresie tym możemy zauważyć, że wartości 1, 2, 9 ani 10 nie są wcale spotykane. Możemy także oszacować, ile mniej więcej było win, które uzyskały poszczególne jakości. Jak widzimy, najwięcej było win, które zostały sklasyfikowane jako wina o jakości 5 i 6.

5.2 Test drugi

SC	MN	WYNIK	SC	MN	WYNIK	SC	MN	WYNIK
0.1	99	46,09130707	0.7	99	62,03877423	1.3	99	63,10193871
0.1	199	47,27954972	0.7	199	64,66541588	1.3	199	66,72920575
0.1	799	84,30268918	0.7	799	81,11319575	1.3	799	82,80175109
0.1	899	90,05628518	0.7	899	83,80237649	1.3	899	85,61601001
0.1	1499	99,62476548	0.7	1499	99,37460913	1.3	1499	99,4996873
0.1	1599	100	0.7	1599	100	1.3	1599	100
0.3	99	62,22639149	0.9	99	63,03939962	1.5	99	63,1644778
0.3	199	66,10381488	0.9	199	64,35272045	1.5	199	65,47842402
0.3	799	83,30206379	0.9	799	82,05128205	1.5	799	82,05128205
0.3	899	84,99061914	0.9	899	83,67729831	1.5	899	84,74046279
0.3	1499	98,37398374	0.9	1499	98,93683552	1.5	1499	99,4996873
0.3	1599	100	0.9	1599	100	1.5	1599	100
0.5	99	62,85178236	1.1	99	63,35209506	1.7	99	63,41463415
0.5	199	67,4796748	1.1	199	66,22889306	1.7	199	67,72983114
0.5	799	80,67542214	1.1	799	82,23889931	1.7	799	82,05128205
0.5	899	83,42714196	1.1	899	84,55284553	1.7	899	84,55284553
0.5	1499	99,4996873	1.1	1499	99,43714822	1.7	1499	99,18699187
0.5	1599	100	1.1	1599	100	1.7	1599	100

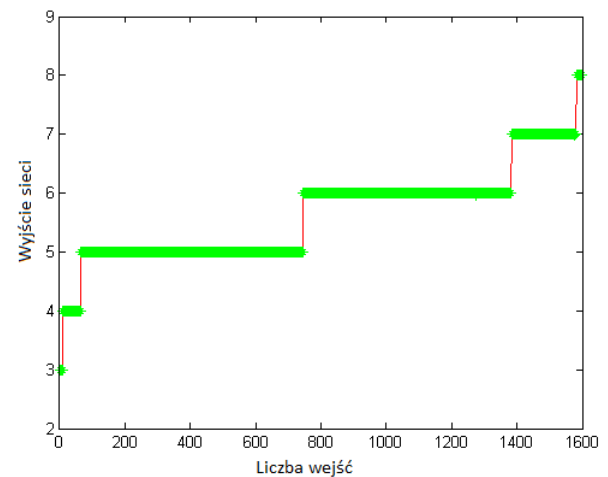
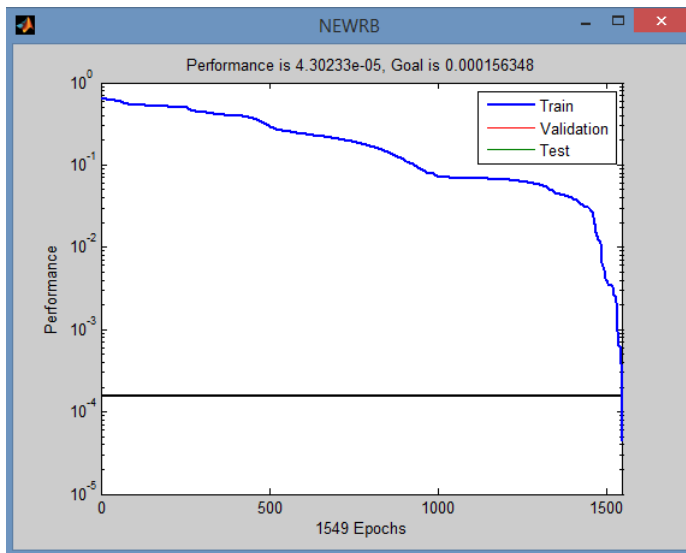
SC	MN	WYNIK
1.9	99	61,97623515
1.9	199	65,79111945
1.9	799	81,8011257
1.9	899	84,17761101
1.9	1499	99,37460913
1.9	1599	100

Celem tego eksperymentu było sprawdzenie zdolności uczących dla wartości **sc** zmieniającej się od 0.1 do 2.0 z krokiem 0.2, oraz liczbie neuronów od 99 do 1599 z krokiem 100.

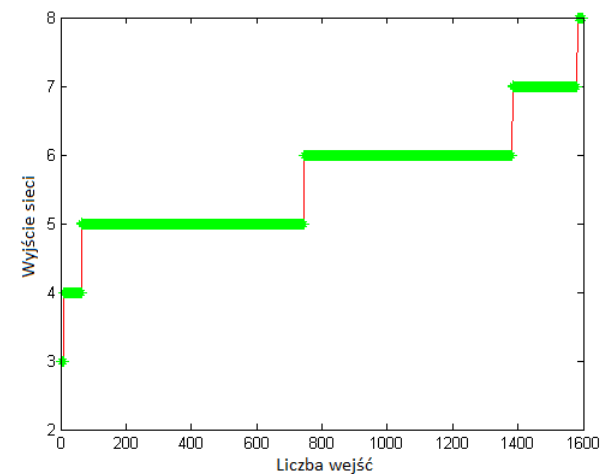
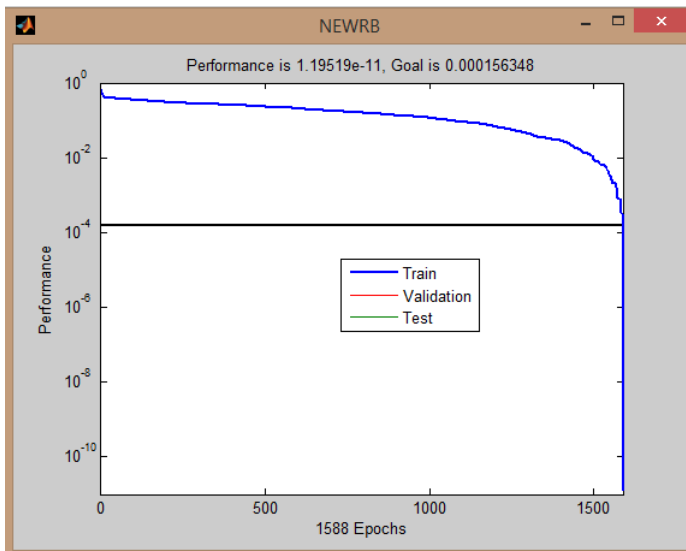
W powyższej tabeli zaprezentowałem wyniki uzyskane dla każdego z testów **sc**, jednak dla liczby neuronów postanowiłem wybrać po 6. Odpowiednio: 2 wyniki dla dwóch pierwszych pomiarów (liczba neuronów: 99 i 199), dwóch środkowych (799 i 899 neuronów) oraz dwóch ostatnich (1499 i 1599 neuronów).

Widzimy, że w każdym przypadku sieć uzyskała 100% poprawnej klasyfikacji dla zakresu 1499-1599 neuronów.

W celu dokładniejszego porównania wyników tego eksperymentu z eksperymentem poprzednim, poniżej wstawiłem wykresy dla dwóch skrajnych pomiarów (dla **sc=0.1** oraz **sc=1.9**).



Rysunek 5: Wykresy pomiaru dla $sc=0.1$.



Rysunek 6: Wykresy pomiaru dla $sc=1.9$.

Jak widzimy na załączonych wykresach, dla pierwszego przypadku poprawne sklasyfikowanie nastąpiło przy 1549 epokach, zaś dla drugiego przy 1588 epokach. Wykresy różnią się od siebie kształtem. Pierwszy wykres ma przebieg nieregularny, zaś drugi swym kształtem przypomina obrócony przebieg funkcji logarytmicznej (przebieg jest gładki).

Tak jak w poprzednim eksperymencie, tak i teraz wartość Performance spadła poniżej wyznaczonego przez nas błędu średniokwadratowego. W pierwszym przypadku spadek ten był nieznaczny, zaś w drugim - sporo większy, jednak mniejszy niż w eksperymencie pierwszym.

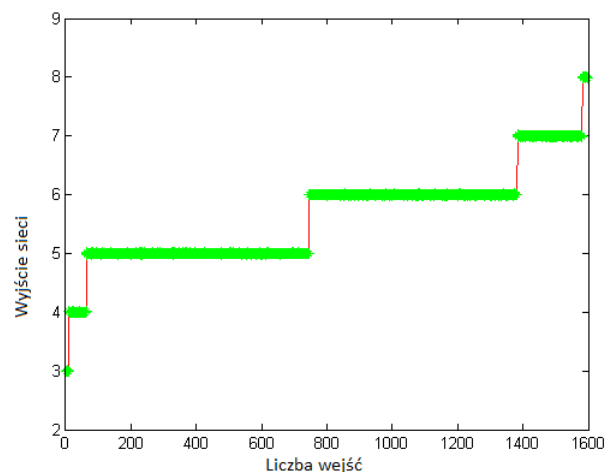
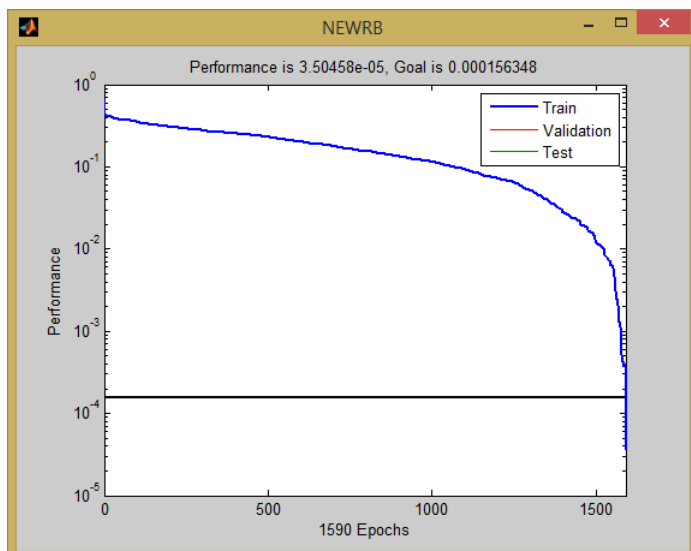
W przypadku wykresów poprawności klasyfikacji procesu nauczania, możemy zauważyć, że one także są podobne do wykresu z poprzedniego eksperymentu. W obu przypadkach nie występują win o jakości 1, 2, 9 ani 10, oraz najwięcej jest win o jakości 5 i 6.

5.3 Test trzeci

SC	MN	WYNIK	SC	MN	WYNIK
3	99	62,72670419	7	99	60,85053158
3	199	65,60350219	7	199	66,54158849
3	799	82,17636023	7	799	81,11319575
3	899	84,30268918	7	899	83,48968105
3	1499	99,12445278	7	1499	97,56097561
3	1599	100	7	1599	97,93621013
4	99	62,6641651	8	99	63,41463415
4	199	66,9793621	8	199	67,97998749
4	799	81,30081301	8	799	83,67729831
4	899	83,80237649	8	899	86,30393996
4	1499	99,24953096	8	1499	95,43464665
4	1599	100	8	1599	95,684803
5	99	62,72670419	9	99	63,10193871
5	199	66,47904941	9	199	67,29205754
5	799	80,98811757	9	799	83,17698562
5	899	84,05253283	9	899	85,36585366
5	1499	99,12445278	9	1499	93,99624765
5	1599	100	9	1599	93,80863039
6	99	62,35146967	10	99	61,16322702
6	199	66,85428393	10	199	66,22889306
6	799	83,98999375	10	799	84,36522827
6	899	86,49155722	10	899	86,86679174
6	1499	99,37460913	10	1499	91,11944966
6	1599	99,68730457	10	1599	91,24452783

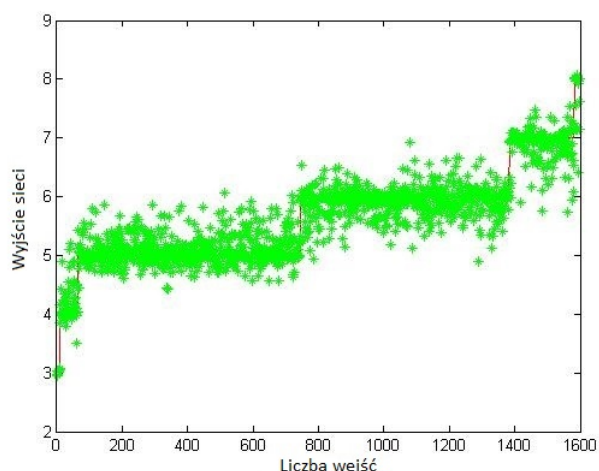
W tym eksperymencie przeprowadziłem testy dla **sc** zmieniającego się od 3 do 10 z krokiem 1. Tak samo jak w eksperymencie poprzednim, tak i teraz, postanowiłem zaprezentować wyniki dla każdego **sc** oraz dla sześciu liczb neuronów.

W powyższej tabeli widzimy, że sieć uzyskała 100% poprawnej klasyfikacji tylko dla $sc = [3, 4, 5]$. Powyżej tej wartości nie było możliwości poprawnego sklasyfikowania wszystkich win. W celu dokładnego określenia liczby epok niezbędnych do uzyskania 100% klasyfikacji w przypadku **sc=5**, zamieściłem poniżej wykresy dla tego pomiaru.



Rysunek 7: Wykresy pomiaru dla $sc=5.0$.

Powyższe wykresy są identyczne jak wykresy z poprzednich eksperymentów, z tą różnicą, że w tym przypadku sieć potrzebowała dokładnie 1590 epok, aby osiągnąć 100% poprawnej klasyfikacji.

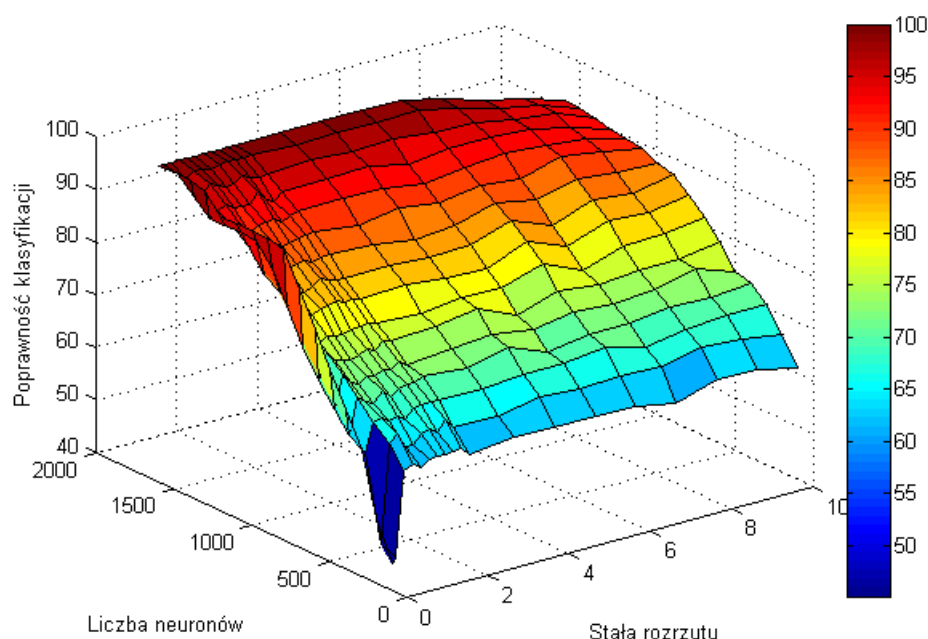


Rysunek 8: Wykres pomiaru dla $sc=10$

Przedstawiony wyżej wykres pokazuje nam efekt niewłaściwej klasyfikacji. Wykres ten został utworzony dla $sc=10$ oraz liczby neuronów równej 1599.

Sieć tutaj wykorzystała wszystkie epoki, a mimo to ostateczny wynik wyniósł: **91,24452783**. Widzimy na tym wykresie, że nasze dane są silnie porozrzucane, co świadczy o niewłaściwej klasyfikacji. Zauważyć można także, że mimo takiego rozrzutu, najprawdopodobniej żadne wino nie zostało sklasyfikowane ani poniżej 3, ani powyżej 8.

Poniżej zamieszczam wykres przedstawiający zależność poprawności klasyfikacji od liczby neuronów i stałej rozrzutu, wykonany dla wszystkich danych.



Rysunek 9: Wykres zależności poprawności klasyfikacji od liczby neuronów i stałej rozrzutu

Na wykresie tym możemy zaobserwować, jak zmieniały się uzyskiwane wyniki eksperymentów w zależności od tego, jakie podaliśmy **sc** oraz **liczbę neuronów**.

Widzimy, że dla małego **sc** (0.01), poprawność klasyfikacji rosła znacznie szybciej, niż w przypadku kolejnych eksperymentów. Na wykresie widać, że dla stałej rozrzutu od 0.01 do 6 wyniki były najlepsze. Dla każdego pomiaru poprawność klasyfikacji wyniosła 100%, oprócz pomiaru ostatniego (**sc**=6). Wówczas poprawność klasyfikacji wyniosła 99,68730457%. Jest to wynik bliski idealnego. Jeżeli wykonamy proste obliczenia: $\frac{1599 \cdot 99,69\%}{100\%}$, to w przybliżeniu uzyskamy 1594, czyli liczbę właściwie sklasyfikowanych win. Po odjęciu daje nam 5, czyli liczbę win źle sklasyfikowanych. Przy stałej rozrzutu powyżej 6 widzimy, że poprawność klasyfikacji znacznie maleje. O ile dla mniejszej liczby neuronów nie jest to aż tak zauważalne, o tyle dla liczby neuronów bliskiej maksymalnej, różnice są już widoczne.

6 WNIOSKI KOŃCOWE

Projekt miał na celu zrealizowanie radialnej sieci neuronowej uczącej się rozpoznawania jakości wina na podstawie różnych danych fizykochemicznych. Do dyspozycji mieliśmy dane dla win białych i czerwonych, lecz w ramach projektu ograniczono się do zrealizowania sieci tylko dla win czerwonych, gdyż proces tworzenia sieci dla tych drugich trwał bardzo długo. Dla porównania: czas tworzenia sieci dla win czerwonych przy wykorzystaniu wszystkich neuronów (1599) trwał około 20 minut, zaś dla win białych (4898 neuronów) czas ten wyniósł ponad 7 godzin. Dlatego zdecydowano się na uszczuplenie liczby danych wejściowych. Możemy z tego wywnioskować, że na czas budowania sieci ma wpływ liczba danych wejściowych. Oprócz tego wpływ mają również:

- Liczba wykorzystanych neuronów (mn),
- Zadany błąd średniokwadratowy (mse),
- Rozrzut radialnej funkcji bazowej (sc).

Najlepszy wynik uzyskano dla $sc=0.01$ oraz liczby neuronów równej 1356. Wtedy czas był relatywnie krótki, jak i poprawność klasyfikacji wyniosła 100%.

W związku z powyższym, możemy powiedzieć, że w tym przypadku nie ma większego sensu (oprócz edukacyjnego), tworzenie sieci dla dużego współczynnika sc , gdyż dla rozpiętości równej 6 i większej, sieć w żadnym przypadku nie uzyskała 100% poprawności klasyfikacji.

Literatura

- [1] Stanisław Osowski, *Sieci neuronowe w ujęciu algorytmicznym*, Wydawnictwa Naukowo-Techniczne, Warszawa 1996
- [2] Stanisław Osowski, *Sieci neuronowe do przetwarzania informacji*, Oficyna Wydawnicza Politechniki Warszawskiej, Warszawa 2006
- [3] https://en.wikipedia.org/wiki/Radial_basis_function_network
- [4] https://pl.wikipedia.org/wiki/Sieć_radialna